

Guide de Validation MLA

Prédiction de Réussite des Étudiants — Questions & Réponses

Le projet en 3 phrases simples

On a un dataset de 395 étudiants avec leurs notes (G1, G2), leurs absences, leur temps de travail... On veut savoir **qui va réussir son examen de rattrapage** et **quel est son profil académique**. Pour ça, on utilise 3 algorithmes ML : PCA (réduction), K-Means (profils), Random Forest (prédiction).

1. Questions sur le Dataset

Q : C'est quoi votre dataset ?

R : Le UCI Student Performance Dataset. C'est un dataset public qui contient les données de 395 étudiants de lycée au Portugal. Chaque étudiant a 33 caractéristiques : son âge, ses notes G1 et G2 (2 premières périodes), ses absences, combien d'heures il travaille par semaine, si ses parents sont ensemble, etc. Et on a aussi G3 (note finale) qu'on utilise pour savoir s'il a réussi ou échoué.

→ $G3 \geq 10$ = réussite | $G3 < 10$ = échec

Q : Pourquoi ce dataset ?

R : Parce qu'il correspond exactement à notre problématique : prédire la réussite académique. Il est bien connu, propre (pas de valeurs manquantes), et contient des données réalistes sur des étudiants. C'est le dataset de référence pour ce type de problème ML.

Q : Il y a des valeurs manquantes ?

R : Non. Le dataset UCI Student Performance est déjà propre — 0 valeurs manquantes. On a quand même fait une vérification dans le notebook avec `df.isnull().sum()`.

Q : C'est quoi votre variable cible ?

R : G3 (la note finale sur 20). On l'a transformée en variable binaire : si $G3 \geq 10$ → réussite (1), si $G3 < 10$ → échec (0). On appelle ça 'pass' dans le code.

→ $pass = 1$ si $G3 \geq 10$, $pass = 0$ sinon

2. Questions sur la PCA (Réduction de Dimensionnalité)

Q : C'est quoi la PCA et pourquoi vous l'utilisez ?

R : PCA = Principal Component Analysis. En simple : on avait 33 variables (features). La PCA les compresse en gardant seulement l'essentiel. Comme quand tu compresses une image JPEG — tu perds un peu de détail mais tu gardes l'essentiel. On l'utilise pour que K-Means et Random Forest travaillent sur des données plus claires et moins bruitées.

Q : Comment vous avez choisi le nombre de composantes ?

R : On a choisi automatiquement en disant : 'garde assez de composantes pour expliquer 95% de la variance'. On n'a pas choisi un chiffre fixe — sklearn le calcule tout seul avec `n_components=0.95`. On a aussi fait un graphique de variance cumulée pour visualiser le coude.

→ `n_components=0.95` dans le code → sklearn choisit automatiquement

Q : Pourquoi 95% de variance ?

R : C'est un seuil standard en ML. 95% veut dire qu'on garde presque toute l'information utile et on supprime seulement 5% qui est souvent du bruit. Si on avait pris 100%, ça ne servirait à rien. Si on avait pris 80%, on perdrait trop d'info.

Q : Est-ce que la PCA améliore vraiment les résultats ?

R : Oui, dans notre cas. Sans PCA, K-Means aurait du mal à bien clustériser 33 variables (problème de la malédiction de la dimensionnalité). Et le Random Forest tourne plus vite avec moins de features.

3. Questions sur le K-Means (Clustering)

Q : C'est quoi le K-Means ?

R : K-Means regroupe les étudiants en K groupes (clusters) selon leurs ressemblances. C'est comme trier des fruits par ressemblance sans étiquette. Il place des centres (centroïdes) et chaque étudiant va dans le groupe dont le centre est le plus proche.

Q : Pourquoi k=3 ?

R : On a utilisé deux méthodes pour choisir k : • Méthode Elbow : on trace l'inertie (= dispersion) pour chaque k. Le 'coude' du graphique est à k=3 — après, ça s'améliore très peu. • Silhouette Score : mesure si les clusters sont bien séparés. k=3 donne le meilleur score. Et logiquement, 3 profils ont du sens : étudiants faibles / moyens / forts.

→ Elbow + Silhouette → les deux confirment k=3

Q : C'est quoi le Silhouette Score ?

R : C'est une note entre -1 et 1 qui mesure si les clusters sont bien formés. Proche de 1 = les étudiants sont bien séparés en groupes distincts. Proche de 0 = les groupes se mélangent. Notre score est positif → clustering acceptable.

Q : C'est quoi les 3 clusters ?

R : • Cluster At-Risk : étudiants avec G1, G2 faibles, beaucoup d'absences, peu de travail • Cluster Average : étudiants dans la moyenne • Cluster High-Performing : étudiants avec G1, G2 élevés, peu d'absences

→ → Utile pour cibler le soutien pédagogique

Q : K-Means c'est supervisé ou non supervisé ?

R : Non supervisé. On ne lui dit pas quelle est la réponse correcte — il trouve lui-même les groupes. C'est la différence avec Random Forest qui est supervisé (on lui donne les réponses pour qu'il apprenne).

4. Questions sur le Random Forest (Classification)

Q : C'est quoi un Random Forest ?

R : C'est une forêt de plusieurs arbres de décision (100 dans notre cas). Chaque arbre donne son avis (réussite ou échec), et le Random Forest prend le vote majoritaire. C'est plus fiable qu'un seul arbre parce que les erreurs individuelles se compensent.

→ 100 arbres votent → le résultat est plus stable et précis

Q : Pourquoi Random Forest et pas un autre algorithme ?

R : • Robuste aux données bruyantes et aux valeurs aberrantes • Fonctionne bien avec des données mixtes (numériques + catégorielles) • Pas besoin de beaucoup de preprocessing • Donne directement une probabilité (pas juste 0 ou 1) • Moins risque d'overfitting qu'un seul Decision Tree

Q : C'est quoi l'Accuracy, F1-Score et ROC-AUC ?

R : • Accuracy : % de bonnes prédictions au total. Ex: 85% = 85 étudiants sur 100 bien classés. • F1-Score : équilibre entre précision et rappel. Important quand les classes sont déséquilibrées. • ROC-AUC : mesure si le modèle distingue bien réussite/échec. 1.0 = parfait, 0.5 = aléatoire. Notre score > 0.90 → excellent.

→ → *On utilise les 3 métriques car chacune mesure un aspect différent*

Q : C'est quoi le train/test split ?

R : On divise les données : 80% pour apprendre (train), 20% pour tester (test). Le modèle apprend sur le train et on mesure ses performances sur le test qu'il n'a jamais vu. On utilise stratify=y pour garder la même proportion réussite/échec dans les deux parties.

→ → *stratify garantit que le split est représentatif*

Q : C'est quoi la matrice de confusion ?

R : Un tableau 2x2 qui montre les 4 cas possibles : • Vrai Positif : prédit réussite → réel réussite ✓ • Vrai Négatif : prédit échec → réel échec ✓ • Faux Positif : prédit réussite → réel échec ✗ • Faux Négatif : prédit échec → réel réussite ✗ Le pire cas pour nous c'est le Faux Négatif (on rate un étudiant qui aurait besoin d'aide).

Q : Est-ce qu'il y a du overfitting ?

R : Non, Random Forest est résistant à l'overfitting de nature (grâce à la diversité des arbres). De plus on mesure les performances sur un test set séparé — si accuracy train ≈ accuracy test, pas d'overfitting.

5. Questions sur l'Intégration dans la Plateforme

Q : Comment le modèle est utilisé dans l'application ?

R : Le modèle est sauvegardé avec joblib (.pkl). Au démarrage de l'API FastAPI, il est chargé en mémoire. Quand un admin ouvre la page Retake Requests, le frontend Angular appelle /api/ml/analyze avec les données de l'étudiant et reçoit la prédiction en temps réel.

→ *Frontend → Gateway (8080) → ML Service (FastAPI) → Réponse JSON*

Q : C'est quoi FastAPI ?

R : Un framework Python pour créer des APIs REST. C'est rapide, moderne, et génère automatiquement une documentation interactive (Swagger). On l'utilise pour exposer les endpoints /predict, /cluster et /analyze.

→ → *Swagger disponible sur <http://localhost:5001/docs>*

Q : Qu'est-ce qui est affiché dans l'interface ?

R : Pour chaque étudiant qui a fait une demande de rattrapage, l'admin voit : • Une barre de progression : % de chance de réussir (ex: 77%) • Un badge de profil : At-Risk / Average / High-Performing Ces infos aident l'admin à prioriser les décisions d'approbation.

6. Questions Pièges à Anticiper

■ **Q : Pourquoi pas de normalisation avant PCA ?**

R : SI — on normalise bien avec StandardScaler AVANT la PCA. La PCA est sensible aux échelles. Sans normalisation, une variable avec de grandes valeurs (ex: absences 0-30) dominerait sur une variable avec de petites valeurs (ex: studytime 1-4).

■ **Q : Pourquoi pas SVM ou réseau de neurones ?**

R : Le Random Forest est suffisant pour 395 données. Les réseaux de neurones nécessitent des milliers de données pour bien fonctionner. Et SVM est moins interprétable. Random Forest + petit dataset = choix classique et efficace.

■ **Q : Le dataset de 395 étudiants c'est pas trop petit ?**

R : Pour du ML classique (pas deep learning), 395 est acceptable. On obtient ~316 pour l'entraînement et ~79 pour le test. C'est suffisant pour Random Forest. On utilise stratify pour garder la représentativité.

■ **Q : Comment vous gérez le déséquilibre des classes ?**

R : Le dataset a ~65% de réussite et ~35% d'échec — léger déséquilibre. On utilise F1-Score (pas juste Accuracy) pour tenir compte de ça. Random Forest avec stratify gère bien ce cas sans besoin de SMOTE ou oversampling.

■ **Q : G1 et G2 sont dans les features — c'est pas de la triche ?**

R : Non, car dans notre contexte c'est réaliste. Avant un examen de rattrapage, on connaît déjà G1 et G2 de l'étudiant. C'est exactement l'info disponible au moment de la décision d'approbation.